

# AI-assisted development and testing

CS639 | Midterm study notes: Lectures 1-4 only | 1 / 3

## Model vs. harness; human judgment

**Model:** produces responses from inputs (Claude is described as text/images in, text out); no file access by itself.

**Harness:** connects the model to tools in an agentic loop: inspect files, propose changes, edit, run commands, and check results. CLI, IDE, and desktop app are interfaces to the harness.

**Human responsibilities:** remove ambiguity with clear goals/constraints/context; verify correctness with execution, tests, and diffs; own architecture to prevent design drift; diagnose failures and redirect. Confidence is not proof. Review unfamiliar APIs, large changes, unexplained "best" claims, and tests whose success you cannot explain.

**Brownfield** means modifying existing software; **greenfield** means starting a new system. For unfamiliar code: overview the structure, trace relevant behavior, question unusual choices, then propose a targeted change. Understanding existing code is a major part of development.

## TypeScript, p5.js, and unit tests

TypeScript shares familiar Java concepts: static types, classes, interfaces, and access modifiers. In p5.js, **setup()** initializes once; **draw()** runs repeatedly (about 60 times/second) to read input, update state, and render.

```
export function setup() { createCanvas(600, 400); }
export function draw() {
  background(20);
  // Read input, update state, draw.
}

export function formatScore(score: number): string {
  return score.toString().padStart(4, "0");
}
// 7 -> "0007"; 12345 -> "12345" (padding never truncates)

import { it, expect } from "vitest";
it("pads a single digit", () => {
  expect(formatScore(7)).toBe("0007");
});
```

**Arrange / Act / Assert:** prepare inputs, execute the behavior, compare the result with an expected outcome. Above: input 7, call `formatScore`, expect "0007". Keep each test focused on an outcome; passing tests only establish the behaviors they check.

## Verification and the local development loop

**test -> format -> lint -> commit -> push.** `npm run test` uses Vitest to check behavior; `npm run format` uses Prettier to normalize formatting; `npm run lint` uses ESLint to detect rule violations. These checks serve different purposes. Run locally to shorten feedback time; local success does not guarantee CI success.

**Diff review:** did only expected files change, does the result match the request, and is a smaller change possible? Automated checks complement human review rather than establishing that the design or requirements are correct.

## Git: recording, sharing, integrating

**clone** copies a repository; **branch** isolates work; **add** stages changes; **commit** records a local snapshot; **push** sends commits to a remote; **fetch** retrieves remote history; **merge** integrates histories. A merge request proposes integration with review and CI; approval does not bypass failing checks.

```
git checkout -b feature/my-change
git add src/changed-file.ts
git commit -m "Fix collision at screen edge"
git push -u origin feature/my-change
# After feature completion, a local merge:
git checkout main
git merge feature/my-change
```

# Context, tools, and merge conflicts

Lectures 2-3 | 2 / 3

## Memory and context loading

**CLAUDE.md** contains explicit instructions, architecture, and conventions; **auto memory** contains assistant-recorded lessons/patterns. `/memory` manages these; `/init` drafts a **CLAUDE.md** that still needs review. Auto memory lives under `~/claude/projects/<project>/memory/`.

**Scopes:** managed policy -> user `~/claude/CLAUDE.md` -> shared project `./CLAUDE.md` -> personal, gitignored `./CLAUDE.local.md`. Nested **CLAUDE.md** files load on demand as relevant files are read. A standalone **SPEC.md** or root **RULES.md** is not automatically active: import/reference it, e.g. **See @SPEC.md for the full feature specification.**

| Command                       | Purpose                                                    |
|-------------------------------|------------------------------------------------------------|
| <code>/help; @filename</code> | Discover available commands; provide precise file context. |
| <code>/compact; /clear</code> | Compress existing context; start a fresh conversation.     |
| <code>/context; /usage</code> | Inspect context occupancy; inspect usage/limits.           |

Context includes system instructions/tools, memory, skill descriptions, and conversation messages. Full skill instructions load on invocation. The slides stress launching from the correct project folder so its configuration is available.

## Rules vs. skills vs. hooks

| Mechanism                                         | Purpose and behavior                                                                                        |
|---------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| <code>.claude/rules/*.md</code>                   | Contextual instructions. No paths: general rule; paths frontmatter: applies to matching files.              |
| <code>.claude/skills/&lt;name&gt;/SKILL.md</code> | Reusable procedure/checklist. Invoke <code>/name</code> or select when relevant.                            |
| Hook in <code>settings.json</code>                | Script triggered by an event; suitable hooks can block actions. Executes independently of model compliance. |

**Key distinction:** instructions are context the model tries to follow; hooks provide event-driven execution. For example, a `PostToolUse` hook matching `Edit|Write` can run `npm run format` after edits. A prose instruction to "always format" is weaker than a configured hook.

Example scope: a rule with paths: `["src/**/*.ts"]` in YAML frontmatter can request tests whenever TypeScript functions change.

## Settings and permissions

User settings: `~/claude/settings.json`; shared project: `.claude/settings.json`; personal/gitignored: `.claude/settings.local.json`. The lecture describes more-specific scope overriding broader scope, with an additional managed-policy scope. **allow** runs silently, **ask** requests approval, **deny** refuses. The **env** section supplies environment variables automatically.

```
{ "permissions": { "allow": ["Bash(npm run test *)"],
  "ask": ["Bash(git push *)"], "deny": ["Bash(curl *)"] },
  "env": { "NODE_ENV": "development" } }
```

## Merge conflict interpretation and resolution

```
<<<<<<< HEAD
export const ENEMY_SPAWN_INTERVAL_MS = 1200;
=====
export const ENEMY_SPAWN_INTERVAL_MS = 600;
>>>>>>> tune/harder-waves
```

In a normal merge, **HEAD** is the current branch; below `=====` is the incoming branch. Git stops because it cannot reconcile the edits automatically. Understand both intentions; choose/combine/rewrite; remove markers; test; stage the resolution; finish the merge commit. **Merge** preserves branch relationships; **rebase** replays commits and rewrites history, potentially removing merge evidence.

# The Elephant-Goldfish Model

Lecture 4 | 3 / 3

## Problem and durable context

Two failure modes: **over-ambition** (too much at once, context runs out) and **premature victory** (declares completion too early). The capable "elephant" needs durable external memory. A fresh "goldfish" session tests whether documentation is sufficient without the earlier conversation.

**Context hierarchy:** summarize deepest source directories first; verify each summary against the code; summarize those summaries upward to the root; refresh when code changes. This avoids dumping an entire codebase into context. Lecture 4 uses nested CLAUDE.md files for automatic loading, refining the earlier "README hierarchy" idea.

**Hierarchy vs. specification:** the hierarchy explains existing code; the feature spec states what to change and why. Durable documents retain decisions across sessions, while the conversation alone does not provide reliable long-term memory.

## Four phases - know the order and purpose

**1. Growing the Elephant:** load relevant context and discuss a prose proposal. **No implementation.** Apply the sycophant challenge: "What could go wrong?" "What edge cases are missing?" "Is there a simpler way?" Challenge the first proposal rather than accepting agreement as evidence.

**2. Teaching the Elephant:** write an implementable design document containing **Problem, Technical Plan, Alternatives, and Detailed Implementation**. Record rejected options and reasons, not just the final choice. Retain a separate feature plan in docs/plans/ rather than overwrite previous plans. Could someone without the discussion build from this document?

**3. Goldfish Protocol:** start fresh with /clear and provide **only the spec + context hierarchy**, not the design conversation. Test **comprehension** (explain the feature back), **criticism** (find gaps), and **readiness** (know exactly what to build). Revise the spec before coding. "Context-free" means free of prior discussion, not free of documentation.

**4. Implementation:** approve the plan; build against the goldfish-tested specification; verify with tests as you go. Finish with a **mean code review**: explicitly look for defects. Generating a solution and critically examining it are different postures.

## Permission modes support phase separation

| Mode         | Behavior as presented in Lecture 4                                         |
|--------------|----------------------------------------------------------------------------|
| Auto         | Default; background classifier reviews actions. Code can still be written. |
| Manual       | Edits and commands require approval.                                       |
| Accept Edits | Edits apply automatically; inspect the diff afterward.                     |
| Plan         | Research and planning; implementation edits blocked until approval.        |

Use **Plan Mode for Phases 1-3** to support the No Code Rule; switch to execution for Phase 4. The deck uses Shift+Tab to cycle modes, /plan to enter planning, and Ctrl+G to open the plan in an editor. Permission mode changes when approval happens, not your responsibility to verify.

## Repeatable procedures and a worked task

Skills can package context-hierarchy creation, design discussion, specification writing, goldfish testing, and critical review. Course examples: /design-discussion, /write-spec, /goldfish-test, /mean-review. These are reusable project procedures, not necessarily built-in commands.

**Review-preview example:** truncate\_review\_text(text, max\_length) must shorten a long review with an ellipsis without splitting a word. Discuss edge cases, document exact behavior, test the spec with a fresh session, then implement and test. A working function alone does not prove the design process or specification is sound.

**Source scope:** distilled from the local lecture 1-4 slide decks; syllabus and administrative details removed. These notes cover the concepts taught there, not a prediction of exam questions or the remaining lectures in the midterm.